

INVIGORATE HEALTH

Invigorate Healthcare App Platform

Engineering Coding Standards

Revision 3.2 — Final Consolidated Edition

Frontend (React / React Native) · Backend (Python FastAPI) · Data & AI · Infrastructure

Applies to all engineering teams — Invigorate Health

Every rule is expressed as a table for fast lookup and CI enforceability. Consolidates the 27-item Change Register and Rev 2.3, and closes the remaining industry gaps: client-side security, authentication & sessions, data classification, the conversational data store with its semantic layer and metagraph, environments & feature flags, caching / jobs / uploads, internationalization, deployment strategy, incident management, third-party licensing, and micro-frontend architecture for the web portals.

Table of Contents

§	Section	Status	Page
1	General Principles	—	3
2	Backend Standards & Database Structure	—	3
3	API Contract	—	4
4	API Response-Time Targets	—	4
5	Frontend, Mobile & Micro-Frontends	REV 3.2	5
6	App Navigation & Frontend Performance	—	7
7	Client-Side Security (Web & Mobile)	NEW 3.0	8
8	Authentication, Tokens & Sessions	NEW 3.0	8
9	Data Classification & Test Data	NEW 3.0	9
10	Security & Compliance	—	9
11	AI & RAG	—	10
12	Conversational Data Store, Semantic Layer & Metagraph	NEW 3.1	10
13	Naming, Comments & Documentation	—	14
14	Configuration, Environments & Feature Flags	NEW 3.0	15
15	Caching, Background Jobs & File Handling	NEW 3.0	15
16	Internationalization & Localization	NEW 3.0	16
17	Delivery, Testing & Platform	—	17
18	Incident Management & Operational Readiness	NEW 3.0	18
19	Third-Party Dependencies & Licensing	NEW 3.0	19
20	Enforcement	—	19

1 General Principles

Principle	Rule
Readability over cleverness	Code must be explicit and easy for any engineer to reason about. No obscure language hacks or over-engineering.
Boy Scout Rule	Leave the codebase cleaner than you found it — refactor messy code, debt, and unoptimised queries safely as you go.
Self-documenting code	Highly descriptive names for variables, functions, components, classes. No abbreviations (const d = data.m...).
Strict typing	Dynamically typed code without explicit declarations is forbidden. TypeScript strict mode; Python type hints on all signatures.
Secure by default	The safe behaviour is the default behaviour — controls are structural (RLS, guarded navigators, default-deny) rather than relying on developer memory.
Grounded by default	No AI-facing surface invents its own definitions. Meaning comes from the semantic layer and metagraph (§12), never from the model's memory.

2 Backend Standards — Python FastAPI

Area	Standard
Architecture	Strict layered pattern: api/ → services/ → repositories/, with core/ (config, Keycloak, middleware), models/ (ORM), schemas/ (Pydantic). Business logic never lives in routes.
Validation	Pydantic v2 models for every request and response body — model_config = ConfigDict(frozen=True). Raw dicts / unvalidated JSON prohibited.
Tenant identity	Derived only from the verified Keycloak JWT via a shared Depends() layer. tenant_id in a request body is prohibited (BOLA vector).
Async	async def for I/O-bound endpoints with async drivers (asyncpg); plain def only for CPU-bound work offloaded to the thread pool.
Errors	Never leak raw exceptions. Custom exception handlers return the standard error contract in §3.
PHI at rest	All wearable/biometric fields are PHI — column-level encryption inside PostgreSQL, no exceptions.

2.1 Database Structure — applies to every table on the platform, including §12

Element	Rule
Primary keys	Surrogate UUID (v7 preferred). No sequential integers on exposed resources. Business keys via UNIQUE constraints only.
Audit columns	Every table: created_at, updated_at — TIMESTAMPTZ NOT NULL, UTC, set by the database (DEFAULT now() / trigger).
Tenant scoping	Tenant-owned tables: tenant_id NOT NULL + FK, index with tenant_id leading, Row-Level Security policy enabled.
Naming	snake_case everywhere; fixed convention pk_<t>, fk_<t>_<col>, uq_, ck_, ix_ — unnamed constraints prohibited.
Nullability & types	NOT NULL by default (nullable needs a stated reason in the migration). NUMERIC for money — never FLOAT. Explicit defaults on booleans/enums.
Deletion	Hard delete is the default. Soft delete (deleted_at) only where clinically/legally required, documented per table, honoured by RLS.

Element	Rule
Migrations	One reversible Alembic migration per PR, hand-reviewed autogenerate, verified rollback plan. Raw SQL prohibited unless formally approved. Expand/contract pattern for zero-downtime deploys (§17).

3 API Contract

Area	Standard
Schema	Every endpoint passes OpenAPI schema review before merge. Backward compatibility within a major version.
Versioning	URL-versioned (/v1, /v2). Breaking change ⇒ new major version + documented deprecation window.
Success shape	Resource under data, request metadata under meta (correlation ID; pagination fields for collections).
Error shape	RFC 9457 Problem Details (application/problem+json): type, title, status, detail, instance + error_code, correlation_id. No PHI, stack traces, or SQL in any error body.
Pagination	Cursor-based only: limit + cursor in; meta.next_cursor + meta.has_more out. Default and max limit declared in the spec. Unbounded collections prohibited.
Concurrency control	Mutable resources return ETag; updates require If-Match and return 412 on mismatch. State-changing endpoints accept an Idempotency-Key so retries never duplicate effects.
Streaming	AI/chat endpoints stream via SSE with a defined event contract (token, citation, tool, done, error) — partial failures surface as an error event, never a silent truncation.
Fields	snake_case JSON, ISO-8601 UTC timestamps, UUID string identifiers, units declared in the schema.
Rate limiting	Per-client and per-tenant limits at the gateway; 429 with Retry-After.

Status	Use
200 / 201 / 204	Read or update · created (with Location header) · delete / no body
400 / 401 / 403	Malformed request · unauthenticated · forbidden (incl. cross-tenant attempts)
404 / 409 / 412 / 422	Not found (also used over 403 where existence is sensitive) · conflict · precondition failed (stale ETag) · semantic validation failure
429 / 5xx	Rate limited (Retry-After) · server fault — never with internal detail

4 API Response-Time Targets

Measured at the gateway, per endpoint class, over a rolling 30 days. p95 is the CI/SLO gate; p99 is the alert line. Sustained breach is treated as a failed non-functional requirement under §20.

Endpoint Class	p95 Target	p99 Target	Notes
Health / readiness (/health, /ready)	≤ 100 ms	≤ 250 ms	No downstream calls; must not touch the database.
Single-resource read (GET /v1/x/{id})	≤ 300 ms	≤ 800 ms	Served from indexed lookup; N+1 queries prohibited.
Collection read / search	≤ 500 ms	≤ 1.2 s	Always cursor-paginated (§3); max limit enforced.

Endpoint Class	p95 Target	p99 Target	Notes
Write (POST / PUT / PATCH / DELETE)	≤ 500 ms	≤ 1.5 s	Slow side-effects go async via events/jobs (§15, §17).
Conversation history read	≤ 400 ms	≤ 1 s	Cursor-paginated over conversation_message (§12.1).
Vector retrieval (top-k)	≤ 400 ms	≤ 900 ms	Excludes generation; measured at the retrieval service.
Wearable batch ingestion	≤ 2 s	≤ 5 s	Batched, idempotent; oversize batches rejected with 422.
AI / RAG generation	first token ≤ 1.5 s	complete ≤ 10 s	Responses must stream; no silent spinner beyond 1.5 s.
Report / export generation	202 accepted ≤ 300 ms	—	Long jobs return 202 + status URL; never a hanging request.

Rule	Standard
Availability SLO	99.9% monthly per service, with error budgets and burn-rate alerting (§17).
Timeouts	Every outbound call has an explicit timeout shorter than the caller's own budget; no unbounded waits.
Load testing	Targets verified in CI load tests against production-like data volumes before each release.

5 Frontend, Mobile & Micro-Frontends — React / React Native

Area	Standard
Components	Functional components + hooks only; class components prohibited. Split files at 250 lines into sub-components / custom hooks.
Server state	React Query (TanStack) for all remote data — fetching, cache invalidation, background refresh, retries. Never dumped into Redux.
Local state	useState for component state; context providers for app-wide theme / language only.
Device feeds	Wearable streams contained in localized custom hooks to prevent tree-wide re-renders.
Lists	Virtualized <FlatList> / <SectionList> with stable keyExtractor. Large dynamic content in raw <ScrollView> prohibited.
Chat rendering	Conversation threads are virtualized and cursor-paginated; streamed tokens append without re-rendering the full thread. Citations render as tappable source chips (§12.5).
Images	react-native-fast-image (or equivalent caching lib) over native <Image> for scrolling content.
Native bridges	HealthKit / Health Connect / Fitbit isolated in independent modules with full permission-denied fallbacks.
Accessibility	WCAG 2.2 AA; axe assertions run in CI (§17).

5.1 Micro-Frontend Architecture [NEW 3.2]

Applies to the web portals (member, coach console, admin). React Native is a single binary — see the React Native row below. A micro-frontend is an independently deployable, team-owned slice of the UI; it is not a licence to fragment the design system, the security model, or the performance budgets, all of which remain platform-wide.

Area	Standard
Decomposition	Micro-frontends are split by business domain (assessments, wearables, conversation, coaching console, admin), never by technical layer. One team owns each micro-frontend end to end, including its backing services and its entry in the service manifest (§17.3).
When not to split	A micro-frontend is justified only by independent deployability and separate team ownership. Splitting for reuse alone is prohibited — shared code ships as a versioned package, not as a remote.
Composition	Web portals compose at runtime via Webpack/Rspack Module Federation behind a thin host shell. Build-time composition (npm-packaged remotes) is the fallback where runtime loading cannot meet §6.2 budgets. Iframe composition is prohibited for PHI surfaces (§7 clickjacking and storage isolation).
Host shell	The shell owns routing, the authenticated navigator group (§6.1), theming, locale, and error/suspense boundaries only. Business logic in the shell is prohibited — a shell release must never be required to ship a domain change.
React Native	Mobile is a single binary: remotes are not loaded over the air. Domain modularity is enforced by package boundaries with an import-linting rule; over-the-air delivery of executable code is prohibited (store policy and §19).
Independent deploy	Each micro-frontend builds, versions (SemVer), and deploys on its own pipeline with its own canary and rollback (§17.1). No coordinated big-bang releases across remotes.
Versioning & compatibility	The host pins a compatible version range per remote and resolves it at deploy time. A remote must remain compatible with the two previously deployed host versions, mirroring the mobile rule in §17.1.

5.2 Micro-Frontend Contracts & Shared Code [NEW 3.2]

Area	Standard
Contracts	Every remote exposes a typed, reviewed contract: exported module names, mount/unmount signature, and props. Contract changes follow §3 rules — backward compatible within a major, breaking change means a new major plus a deprecation window.
Cross-app state	Remotes never reach into each other's stores. Communication is via the host-provided context (auth, tenant, locale, theme) or a typed, versioned browser event bus. Direct imports between remotes are lint-blocked.
Server state	React Query cache is per micro-frontend (§5). A shared query client across remotes is prohibited — cross-remote invalidation goes through the event bus.
Shared dependencies	React, React DOM, React Query, the design system, and the i18n runtime are declared singletons with the version range pinned in one shared config. Duplicate React instances fail the build.
Design system	One versioned design-system package is the only sanctioned source of UI primitives and tokens. Forked or per-remote component libraries are prohibited (WCAG 2.2 AA consistency, §5).
Routing	Route names stay in the central typed registry (§6.1); each remote declares its owned route prefix. Two remotes claiming the same prefix is a build failure.
i18n	Each remote ships its own namespaced locale bundles (§16), loaded lazily by the shared i18n runtime; keys are namespaced by remote to prevent collisions.

5.3 Micro-Frontend Runtime, Security & Testing [NEW 3.2]

Area	Standard
Performance budget	The host shell's initial route stays within the §6.2 budget (≤ 250 KB gzipped) with remotes loaded lazily per route. Each remote carries its own entry budget of ≤ 150 KB gzipped, enforced in CI.

Area	Standard
Failure isolation	Every remote mounts inside an error boundary and a loading boundary. A remote that fails to load or throws degrades to a scoped fallback with retry — it never blanks the shell or blocks navigation.
Load timeouts	Remote loading has an explicit timeout (§4 timeout rule); on expiry the shell renders the fallback rather than waiting. Skeleton loaders apply as in §6.1.
Security	Remote bundles are served from an allowlisted origin with Subresource Integrity and are covered by the strict CSP in §7 — script-src must not be widened to host a remote. Remotes inherit the §7 storage rules: no tokens or PHI in web storage, access tokens stay in host memory.
Authorization	Authorization is enforced server-side per §2 and §8. Mounting or hiding a remote is presentation only and is never the access-control boundary.
Data classification	T1 conversation and health surfaces (§9) follow the same rules inside a remote as anywhere else — no PHI in route params (§6.1), analytics, or logs.
Observability	Every remote emits its bundle name and version on OpenTelemetry traces and error reports (§17.3), so a frontend incident resolves to an exact remote version in the deploy timeline (§18).
Testing	Each remote meets the §17.2 UI gate (80% coverage plus axe assertions) in its own pipeline, and contract tests against the host run on both sides of the boundary. An end-to-end suite exercises the composed application before production promotion.

6 App Navigation & Frontend Performance

6.1 Navigation Standards

Area	Standard
Library	React Navigation (native-stack) is the single sanctioned navigator; one root navigator per app, no ad-hoc custom routers.
Route registry	All route names PascalCase, declared once in a central typed registry with typed params (RootStackParamList). String-literal navigation calls prohibited.
Structure	Auth flow and app flow are separate navigator groups; protected screens live only inside the authenticated group — guards are structural, not per-screen ifs.
Deep linking	Every screen addressable via universal links (https) + app scheme, mapped in one linking config. Deep links into protected screens re-authenticate first.
Params & PHI	Route params carry identifiers only (UUIDs) — never PHI, tokens, or query-strings with personal data; screens fetch their own data by ID.
Back behaviour	Android hardware back always matches the visible back affordance. Destructive/multi-step flows confirm before discarding state; back never re-submits.
State restoration	Navigation state persists across process death; returning users land on their last safe screen (never a stale PHI view after session expiry).
Loading pattern	Skeleton loaders on navigation, not blank screens or full-screen spinners; paginated lists use cursor-driven infinite scroll (§3) with an end-of-list state.

6.2 Frontend Performance Budgets

Metric	Budget	Applies To
Cold start → first interactive screen	≤ 2 s	Mobile app
Warm start	≤ 500 ms	Mobile app
Screen-to-screen navigation transition	≤ 300 ms	Mobile app

Metric	Budget	Applies To
List scrolling	60 fps, no dropped-frame jank on virtualized lists	Mobile app
LCP (Largest Contentful Paint)	≤ 2.5 s	Web portals
INP (Interaction to Next Paint)	≤ 200 ms	Web portals
CLS (Cumulative Layout Shift)	≤ 0.1	Web portals
JS bundle (initial route)	≤ 250 KB gzipped; code-split beyond it	Web portals
Time to first byte (TTFB)	<800ms	
TTFB	< 1.8 s	
Visual Complete 85%	< 3.0 s	
Visual Complete 95%	< 4.0 s	

7 Client-Side Security (Web & Mobile) [NEW 3.0]

Area	Standard
XSS	dangerouslySetInnerHTML, eval, and dynamic Function() are prohibited. Any rendered rich text (knowledge content, ebooks, LLM output) is sanitized with DOMPurify against an allowlist.
Content Security Policy	Web portals ship a strict CSP: no unsafe-inline / unsafe-eval, script-src limited to self + hashed/nonced scripts, frame-ancestors 'none' (clickjacking).
CSRF	Session cookies are Secure, HttpOnly, SameSite=Strict; state changes never ride on GET; token-header pattern for API calls from browsers.
Web storage	No PHI, tokens, or secrets in localStorage / sessionStorage / IndexedDB. Access tokens live in memory; refresh via HttpOnly cookie.
Input trust	Client-side validation exists for UX only — every input is re-validated server-side (§2). The client is never a trust boundary.
Third-party scripts	Minimized, individually approved (§19), loaded with Subresource Integrity hashes; no third-party script on PHI-bearing pages.
Security headers	HSTS (includeSubDomains, preload), X-Content-Type-Options: nosniff, Referrer-Policy: strict-origin-when-cross-origin on all web responses.
Screen privacy	Mobile PHI and conversation screens set FLAG_SECURE / snapshot blurring so content never appears in app-switcher screenshots or screen recordings.

8 Authentication, Tokens & Sessions [NEW 3.0]

Area	Standard
Identity provider	Keycloak is the single IdP. Hand-rolled authentication, password storage, or token issuance code is prohibited.
Access tokens	JWT, TTL ≤ 15 minutes, audience- and issuer-validated on every request; signing keys rotated per §10.
Refresh tokens	TTL ≤ 30 days, rotating on every use with reuse detection — a replayed refresh token revokes the whole session family.

Area	Standard
Logout & revocation	Logout revokes server-side (Keycloak session + refresh family), never just deletes the client copy. Admin-forced revocation propagates ≤ 60 s.
Session inactivity	PHI-viewing apps re-authenticate after 15 minutes of inactivity (HIPAA automatic logoff); background → foreground beyond the window requires biometric/PIN re-entry on mobile.
Passwords	NIST 800-63B: minimum 12 characters, length over composition rules, checked against breached-password lists, no forced periodic rotation, rate-limited attempts.
Login hardening	Progressive lockout / throttling on failed logins; error messages never reveal whether an account exists (no user enumeration).
MFA	Mandatory for all staff, coach, and admin portals (SOC 2); offered to end users. TOTP or platform passkeys — SMS only as fallback.
Service-to-service	Workload identity (OAuth2 client credentials / K8s service accounts) — no shared static API keys between internal services.

9 Data Classification & Test Data [NEW 3.0]

Every field, event, log line, cache entry, and conversation message carries exactly one classification. Handling rules follow the tier, not developer judgement.

Tier	Examples	Handling
T1 — PHI	Health metrics, assessments, care plans, wearable data, conversation messages, and any embedding derived from them	Column-level encryption at rest; RLS; access logged; never in logs, analytics, URLs, push bodies, or shared caches (§15); erasure per §10 and §12.6.
T2 — PII	Name, email, phone, demographic data	Encrypted at rest; access-controlled; redacted from logs; minimized in third-party tools.
T3 — Internal	Configs, metrics, service telemetry, business reports	Access-controlled; no external sharing without approval.
T4 — Public	Marketing content, published knowledge articles, ontology definitions	No restrictions; still versioned and reviewed.

Rule	Standard
Analytics events	Product analytics and crash reports carry T3 data only — event schemas are reviewed to prove no T1/T2 fields (including message text) leak into third-party SDKs.
Test data	Production PHI never enters non-production environments. Test fixtures are synthetic or de-identified per HIPAA Safe Harbor (all 18 identifiers removed).
Push notifications	Notification payloads contain no T1/T2 content — they reference; the app fetches the sensitive detail after unlock.

10 Security & Compliance

Area	Standard
Frameworks	OWASP ASVS + OWASP API Top 10 2023 — explicit focus on BOLA, auth failures, rate-limit gaps, excessive data exposure.
Transport	TLS 1.2 minimum, TLS 1.3 preferred; all PHI encrypted in transit and at rest.
Audit logs	Append-only / immutable, retained ≥ 6 years (HIPAA §164.316(b)(2)); track entry, modification, and access.

Area	Standard
Log redaction	Never log PHI, tokens, passwords, raw health metrics, JWTs, patient identifiers, or conversation content. Structured logs + correlation IDs everywhere.
Secrets	Approved secret manager only — nothing in code, Git, bundles, logs, or .env.example. Rotation: API keys & DB creds 90d; certs ≤ 90d automated (ACME); JWT signing keys 180d overlapping.
Tenant isolation	JWT-derived tenancy (§2) + PostgreSQL RLS (§2.1) — two independent layers, both mandatory, including every table in §12.
GDPR	Access, erasure, portability, consent versioning, minimization, purpose limitation, automated retention expiry. Erasure covers conversations and derived embeddings; HIPAA retention holds take documented precedence.
Mobile hardening	Keychain / Keystore storage; SSL pinning to intermediate CA or public key with a backup pin; jailbreak/root detection; no PHI in plaintext local storage.
SOC 2	RBAC + MFA; mandatory PR review with immutable audit trail; security monitoring + incident response (§18); periodic subprocessor reviews (§19).

11 AI & RAG

Area	Standard
Prompt safety	Input sanitization + strict role boundaries against prompt injection. Retrieved documents and tool output are untrusted data — never treated as instructions.
Grounding	Answers must cite retrieved source context; ungrounded generation is a failure, not a degraded mode. Enforcement detail in §12.5.
PHI boundary	Automated PHI redaction before any external model call — verified by CI tests — plus a signed HIPAA BAA with every external provider.
Quality	Continuous eval datasets; human review mandated for all generated medical/lifestyle guidance.
Governance	Model, prompt, and evaluation registries; versioned + rollback-capable models; bias testing; drift monitoring.
Clinical boundary	Written determination whether output is clinical decision support; if yes, FDA SaMD / EU MDR assessment with legal input before release.

12 Conversational Data Store, Semantic Layer & Metagraph [NEW 3.1]

The conversation store is the platform's memory; the semantic layer is its dictionary; the metagraph is its map of how things relate. Together they define the only facts an LLM is permitted to assert. A model may generate language freely — it may not generate meaning.

12.1 Conversation Store — Required Tables

PostgreSQL is the system of record for conversations. All tables inherit §2.1 (UUIDv7 PK, tenant_id + RLS, TIMESTAMPTZ audit columns, named constraints). All message content is T1 (§9).

Table	Purpose	Key Relationships
conversation_session	One dialogue thread between a user and an assistant persona.	→ user, tenant, assistant_persona
conversation_message	One turn (user / assistant / system / tool). Append-only.	→ conversation_session; self-ref parent_message_id
message_retrieval_trace	What was retrieved to answer a turn, and how.	→ conversation_message

Table	Purpose	Key Relationships
message_citation	Exact source spans the answer is grounded in.	→ conversation_message, knowledge_chunk
message_tool_call	Each tool/function invocation with arguments and result status.	→ conversation_message
message_entity_link	Entities the turn resolved to, in the metagraph and terminology service.	→ conversation_message, graph_node
conversation_summary	Rolling summary used for context compaction; versioned, never destructive.	→ conversation_session
message_feedback	User rating / flag and reviewer disposition.	→ conversation_message
conversation_safety_event	Guardrail trips: injection, PHI leak, refusal, escalation.	→ conversation_message
knowledge_document / knowledge_chunk	Source content and its retrievable chunks with vector embeddings.	chunk → document; chunk → graph_node

12.2 Attribute Definitions — conversation_session

Column	Type	Rule
session_id	UUID (v7) PK	Surrogate key; exposed identifier.
tenant_id	UUID NOT NULL	FK to tenants; leading column of every index; RLS policy key.
user_id	UUID NOT NULL	FK to users; owner of the thread.
assistant_persona_id	UUID NOT NULL	FK to the registered persona (coaching, knowledge, triage) — personas are registered config, not free text.
title	TEXT NULL	Generated label; T1 — never used in logs, analytics, or notifications.
channel	ENUM NOT NULL	mobile web coach_console. Explicit default required.
status	ENUM NOT NULL	active archived closed. State transitions are one-way and audited.
consent_version_id	UUID NOT NULL	FK to the consent record in force when the thread opened (§10 GDPR).
retention_policy_id	UUID NOT NULL	FK to the retention rule governing expiry; no thread exists without one.
created_at / updated_at	TIMESTAMPTZ NOT NULL	UTC, database-set (§2.1).
deleted_at	TIMESTAMPTZ NULL	Soft delete permitted here (clinical/legal retention); honoured by RLS and every repository query.

12.3 Attribute Definitions — conversation_message

Append-only. Corrections are new rows superseding old ones — an audited conversation may never be edited in place.

Column	Type	Rule
message_id	UUID (v7) PK	Ordering is by v7 key plus created_at, never by client-supplied sequence.
session_id	UUID NOT NULL	FK to conversation_session; indexed (tenant_id, session_id, created_at).
tenant_id	UUID NOT NULL	Denormalized for RLS and index locality — mandatory, not optional.

Column	Type	Rule
parent_message_id	UUID NULL	Self-reference enabling regeneration branches and A/B comparison.
role	ENUM NOT NULL	user assistant system tool. Validated against the enum — no free-text roles.
content	TEXT NOT NULL	T1, column-encrypted. Never logged, cached in shared caches, or sent to analytics.
content_format	ENUM NOT NULL	plain markdown. Rendered only after sanitization (§7).
model_id / model_version	TEXT NOT NULL (assistant rows)	FK-equivalent to the model registry (§11) — every generated turn is reproducible to an exact model version.
prompt_template_id / version	UUID / TEXT	FK to the prompt registry; ad-hoc production prompts are prohibited.
semantic_layer_version	TEXT NOT NULL	Version of the semantic layer used — answers are auditable against the definitions in force at the time.
ontology_version	TEXT NOT NULL	Metagraph version used for entity resolution (§12.4).
grounding_status	ENUM NOT NULL	grounded partially_grounding refused ungrounded. Written by the grounding check, not by the model.
groundedness_score	NUMERIC(4,3) NULL	Automated support score; thresholds in §12.5. NUMERIC, never FLOAT.
token_input / token_output	INTEGER NOT NULL	Cost and abuse monitoring.
latency_ms	INTEGER NOT NULL	Measured against §4 AI targets.
redaction_applied	BOOLEAN NOT NULL	True when PHI redaction ran before an external model call (§11); CI-tested.
created_at	TIMESTAMPZ NOT NULL	UTC, database-set. No updated_at — rows are immutable.

12.4 Semantic Layer — the platform's single dictionary

Every term, metric, and entity an LLM may reference is defined once, versioned, and owned. If a concept is not in the semantic layer, the assistant may not assert it — this is the primary hallucination control.

Table	Purpose
semantic_term	Canonical concept: name, definition, synonyms, owner, status, version. The only vocabulary the assistant may use authoritatively.
semantic_metric	Computed measure: formula/SQL expression, unit, grain, filters, valid range, clinical caveats, owner. Metrics are executed — never estimated in prose by the model.
semantic_dimension	Legal grouping/slicing attributes for each metric (time, cohort, program, device).
semantic_mapping	Binding from term/metric to physical columns in PostgreSQL, openEHR paths, and terminology codes.
semantic_version	Immutable version record with effective dates; referenced by every generated message (§12.3).

Attribute	Type	Rule
term_id / metric_id	UUID (v7) PK	Stable identifier referenced by prompts and citations.
canonical_name	TEXT NOT NULL	Unique per version (uq_semantic_term_name_version). Human-readable, unambiguous.

Attribute	Type	Rule
definition	TEXT NOT NULL	Plain-language definition the assistant is required to use verbatim in meaning — no paraphrasing that changes scope.
synonyms	TEXT[] NOT NULL	Accepted user phrasings mapped to this concept; drives entity resolution.
formula	TEXT NOT NULL (metrics)	Executable expression; the assistant calls it, never re-derives it.
unit / value_range	TEXT / NUMERIC	Unit is mandatory (§3 field conventions); out-of-range results are rejected, not narrated.
clinical_caveat	TEXT NULL	Mandatory qualifier the assistant must surface with the value (e.g. device-measured, not diagnostic).
owner / steward	UUID NOT NULL	Business owner and data steward (§17 governance) — no ownerless definitions.
status	ENUM NOT NULL	draft approved deprecated. Only approved terms are exposed to the model.
version / effective_from / effective_to	TEXT / TIMESTAMPTZ	Definitions change by version; historical answers stay explainable.

12.5 Metagraph — the map of how concepts relate

The metagraph turns isolated definitions into navigable structure, so retrieval follows real relationships instead of vector similarity alone — the second hallucination control.

Element	Definition
graph_node	A typed entity: Condition, Symptom, Intervention, Program, Nutrient, Activity, Device, Metric, ContentAsset, Goal. Every node carries node_type, canonical_name, semantic_term_id, ontology_version, and terminology codes (ICD-10, SNOMED CT, LOINC, MeSH).
graph_edge	A typed, directed relationship: INDICATES, TREATS, CONTRAINDICATES, MEASURED_BY, PART_OF, RECOMMENDED_FOR, DERIVED_FROM. Carries edge_type, confidence, evidence_level, source_reference, and valid_from / valid_to.
Provenance	Every edge cites its evidence (guideline, study, internal clinical review) with a reviewer and date. Edges without provenance are rejected at ingestion — an unsourced relationship is a hallucination waiting to be retrieved.
Contraindication edges	CONTRAINDICATES is a hard blocking edge: a recommendation traversal that crosses one is suppressed and logged as a safety event, regardless of model confidence.
Chunk binding	Every knowledge_chunk links to the graph nodes it discusses, so retrieval can be constrained to a relevant subgraph before ranking.
Versioning & governance	ontology_version is immutable per release; changes require business owner + data steward approval (§17) and re-run of the eval suite before promotion.

12.6 Anti-Hallucination Contract — mandatory for every generated turn

Control	Rule
Closed-world retrieval	The assistant answers only from retrieved, tenant-scoped context plus semantic-layer definitions. Parametric model memory is not an accepted source for any health, program, or metric claim.
Citation required	Every factual sentence maps to at least one message_citation row (chunk + character span). A turn with unmapped factual claims is blocked before delivery.
No arithmetic in prose	Any user metric, trend, or comparison comes from executing a semantic_metric against the database. The model may narrate a returned value; it may never compute or approximate one.

Control	Rule
Entity resolution first	User phrasing resolves to graph_node / semantic_term via synonyms before retrieval. Unresolvable entities trigger a clarifying question — never a best guess.
Groundedness threshold	groundedness_score below the configured floor forces grounding_status = refused and a safe fallback response offering human escalation.
Explicit unknowns	“I don't have that information” is a required, tested response path. Absence of data is never filled with plausible text.
Freshness & scope	Retrieved chunks carry effective dates; expired or out-of-scope content is excluded and the limitation stated to the user.
Injection containment	Retrieved content and tool results are untrusted data (§11). Instruction-shaped text inside retrieved content is neutralized and logged as a safety event.
Traceability	Every answer is reproducible from stored model version, prompt version, semantic_layer_version, ontology_version, and retrieval trace — an unreproducible answer is a defect.
Eval gates	Groundedness, citation-accuracy, refusal-correctness, and contraindication-safety suites run in CI; regression blocks release (§17).

12.7 Retention, Privacy & Isolation

Area	Standard
Classification	Conversation content and its embeddings are T1 PHI (§9) — encrypted, RLS-scoped, access-logged, excluded from logs and analytics.
Erasure	A GDPR erasure request removes messages, summaries, retrieval traces, feedback, and derived embeddings across PostgreSQL and the Vector DB, unless a documented HIPAA retention hold applies (§10).
Retention	Every session is bound to a retention_policy_id with automated expiry; indefinite retention by default is prohibited.
Training use	Conversation data is never used for model training or fine-tuning without explicit, versioned, purpose-specific consent and de-identification review.
Cross-tenant safety	Retrieval, summaries, caches, and eval datasets are tenant-scoped end to end; a shared cache key without tenant_id is a release blocker.
Human escalation	Coach/clinician access to a user's conversation is role-gated, consent-checked, time-boxed, and written to the immutable audit log.

13 Naming, Comments & Documentation

Context	Style	Example
Folders & directories	kebab-case	knowledge-parser/, auth-service/
Classes / TS interfaces	PascalCase	interface UserPayload, class DeviceDataSync
React components	PascalCase	function HealthDashboard() {}
Methods & variables	camelCase (FE) / snake_case (BE)	const userPulse = 72; user_pulse = 72
Constants & env vars	UPPER_SNAKE_CASE	MAX_UPLOAD_LIMIT_MB = 50
DB tables & columns	snake_case	tenant_id, conversation_message
Routes / screens	PascalCase, central registry	AssessmentDetail, WearableSync
Graph node / edge types	PascalCase / UPPER_SNAKE	Condition, Intervention · TREATS, CONTRAINDICATES
i18n keys	dot.notation namespaces	assessment.summary.title

Area	Standard
Comments	Explain why, never what — intent, constraints, regulatory reasons, trade-offs. Restating the code is a review-blocking smell.
Python docstrings	Google style on every public module/class/function (purpose, Args, Returns, Raises) — enforced via Ruff D rules.
TypeScript	TSDoc on every exported function, hook, and prop interface; non-obvious props document units and ranges.
Semantic definitions	Every semantic_term, semantic_metric, and graph_edge type carries a written definition and owner — documentation is data here, not prose (§12.4).
TODO / FIXME	Must reference a card — TODO(INV-123): — lint-flagged; untracked TODOs block merge.
Repo docs	README (purpose, setup, env var names, test commands, manifest link) + ADRs in /docs/ adr + runbook link (§18).
Dead code	Deleted, never commented out — Git history is the archive.

14 Configuration, Environments & Feature Flags [NEW 3.0]

Area	Standard
Twelve-factor config	All configuration comes from the environment (injected by the secret manager / deploy platform). One immutable build artifact is promoted dev → staging → prod — never rebuilt per environment.
Environment parity	Staging mirrors production topology (K8s, gateway, RLS, replication) with synthetic data (§9). Environment-conditional code paths (if env == "prod") are prohibited.
Environment count	Exactly: local, CI (ephemeral), staging, production. Long-lived snowflake environments are prohibited.
Feature flags	Central flag service with typed flag definitions; every risky or progressive rollout ships behind a flag with a kill switch — including model, prompt, and ontology version rollouts.
Flag hygiene	Every flag is created with an owner and a removal ticket; flags older than 90 days past full rollout fail a CI audit. Flags are configuration, not authorization — access control never depends on a flag.
Runtime config changes	Flag flips and config changes are audited (who, what, when) and visible in the deploy timeline for incident correlation (§18).

15 Caching, Background Jobs & File Handling [NEW 3.0]

15.1 Caching

Area	Standard
Cache platform	Redis is the sanctioned shared cache. Keys are namespaced service:tenant:entity:id; every key sets an explicit TTL — no immortal cache entries.
Invalidation	Event-driven invalidation on the platform event bus (§17); TTL is the backstop, not the strategy.
PHI in cache	T1 data (§9) in shared caches only when encrypted, tenant-scoped in the key, and TTL ≤ 5 minutes. Conversation content and embeddings are never cached cross-tenant.
HTTP caching	ETag + Cache-Control on cacheable GETs; authenticated PHI responses are Cache-Control: private, no-store. CDN caches public (T4) content only.

15.2 Background Jobs & Scheduled Work

Area	Standard
Queue-backed jobs	Anything slower than the §4 write budget runs as a queued job (embedding generation, ingestion, exports, summarization), with bounded retries + backoff, idempotent handlers, per-job timeout, and a DLQ with alerting.
Scheduled jobs	Cron-style work runs under a distributed lock so scaled replicas never double-run; every schedule has an owner and a missed-run alert.
Graceful shutdown	All services honour SIGTERM: stop accepting work, drain in-flight requests/jobs ≤ 30 s, then exit — required for zero-downtime deploys (§17).

15.3 File Uploads & Media

Area	Standard
Limits & validation	MAX_UPLOAD_LIMIT_MB enforced server-side; content type verified by sniffing magic bytes — the client-declared MIME type is never trusted; allowlist per endpoint.
Malware scanning	Every uploaded file is scanned before it becomes readable by any user or service.
Metadata stripping	EXIF / embedded metadata (GPS, device IDs) stripped from images on ingestion.
Storage access	Object storage only, private by default, accessed via short-lived presigned URLs scoped to the tenant. Publicly listable buckets are prohibited.
Documents & ebooks	Parsed in an isolated service (no shared credentials) — malformed-file exploits must not reach core services. Parsed chunks enter the knowledge store with provenance (§12.5).

16 Internationalization & Localization [NEW 3.0]

Area	Standard
String extraction	All user-facing strings live in locale resource files (i18next / react-i18next); hardcoded UI strings are lint-blocked.
Message format	ICU MessageFormat for pluralization, gender, and interpolation — no string concatenation of translated fragments.
Locale formatting	Dates, numbers, and units render via Intl APIs per user locale. Measurements stored in SI units and converted at display.
RTL	Layouts verified in RTL for supported locales; direction-sensitive styles use logical properties (start/end), not left/right.

Area	Standard
Assistant language	The assistant responds in the user's locale using approved localized semantic-layer definitions — it never translates a clinical definition on the fly (§12.4).
Medical content	Translated medical / lifestyle guidance requires clinical review in the target language — machine translation alone is prohibited.
Fallbacks	Missing translations fall back to the default locale and are reported — never rendered as raw keys to users.

17 Delivery, Testing & Platform

Area	Standard
Branching	GitHub Flow: feature/INV-### → PR + review → main → automated CI deploy. No direct pushes to main.
Commits	Conventional Commits — type(scope): summary [INV-###] — enforced by commitlint; drives the changelog.
PR size	Target ≤ 400 changed lines per PR; larger PRs require a stated justification and are split where feasible.
Reviews	PHI / auth / tenancy / semantic-layer / ontology changes: two reviewers (one Senior Engineer; semantic and ontology changes also require the data steward). Everything else: one Senior Engineer.
CI gates	Lint → type check → tests + security scans → AI eval suites (§12.6) → merge. Any failed gate locks the PR.
Scanning	Dependabot + Snyk + npm audit + container scanning; Semgrep + Bandit SAST; SBOM generated per build.
Dependencies	Lockfiles committed; --frozen / npm ci builds; unpinned ranges prohibited. Licensing policy in §19.
Releases	SemVer, signed immutable tags, Keep-a-Changelog generated from commits. Model, prompt, semantic-layer, and ontology versions are released and rollback-capable like code.

17.1 Deployment Strategy

Area	Standard
Progressive delivery	Production deploys are canary or blue-green — never big-bang. Automated rollback triggers on SLO burn-rate or error-rate breach during rollout.
Zero-downtime migrations	Schema changes follow expand/contract: additive migration deploys first, code follows, contraction only after the old path is retired.
Rollback	Every release is rollback-capable within the RTO; rollback is rehearsed, not theoretical.
Mobile releases	Staged rollouts with crash-rate gates; server compatibility maintained for the two previous app versions.

17.2 Test Coverage Gates

Component Class	Min Coverage	Additional Gate
Core services	90%	Mutation score ≥ 60%
Shared libraries	95%	Mutation score ≥ 70%
AI components	85%	Groundedness, citation-accuracy, refusal & contraindication suites (§12.6)

Component Class	Min Coverage	Additional Gate
UI components	80%	axe accessibility assertions

17.3 Platform & Architecture

Area	Standard
Microservices	Service manifest in every repo root (owner, domain, API contract, health/ready/live endpoints, dependencies, database owner).
Resilience	Circuit breakers, bounded retries with backoff + jitter (idempotent ops only), explicit timeouts, idempotency keys on state changes.
Events	Versioned envelope (event_id, type, version, tenant_id, correlation_id, timestamp, schema_uri); schema registry; idempotent consumers; DLQs with alerting.
Kubernetes	Requests + limits, HPA, readiness/liveness probes, non-root + read-only FS, restricted PSS, default-deny NetworkPolicies.
IaC	Terraform only, PR-reviewed; remote state with locking; scheduled drift detection in CI.
Observability	OpenTelemetry traces/metrics/logs; Trace, Span, Correlation & Tenant IDs on every request; SLOs + error budgets with burn-rate alerts.
Disaster recovery	RPO ≤ 15 min via continuous replication + PITR; RTO ≤ 4 h; cross-region snapshots; quarterly evidenced restore drills covering the conversation store and Vector DB.
Data ownership	PostgreSQL (users, tenants, RBAC, conversations, semantic layer), openEHR (clinical), Vector DB (embeddings — T1-classified, erasure-compliant), Graph store (metagraph), Terminology (ICD-10, SNOMED CT, LOINC, MeSH). No cross-boundary writes.
Semantic governance	Business owner + data steward accountable for the semantic layer and ontology; every version change is approved, evaluated, and traceable.

18 Incident Management & Operational Readiness [NEW 3.0]

Severity	Definition	Response
SEV1	Suspected PHI exposure, tenant-isolation breach, unsafe clinical output at scale, or platform down	Acknowledge ≤ 15 min, 24/7; incident commander assigned; executive + privacy/DPO notified immediately; HIPAA breach-notification assessment starts at once (60-day statutory clock).
SEV2	Major feature down or SLO hard-breached, no data exposure	Acknowledge ≤ 30 min; status page updated; fix or mitigate before next business day.
SEV3	Degraded performance / partial feature loss with workaround	Acknowledge ≤ 4 business hours; scheduled fix.
SEV4	Cosmetic or low-impact defect	Triaged into the normal backlog.

Area	Standard
On-call	Every production service has a named owning team with an on-call rotation; escalation paths documented in the service manifest.
Runbooks	Each service maintains a runbook (common failures, dashboards, rollback steps, dependency contacts) linked from its manifest — verified during DR drills.
Postmortems	Blameless postmortem within 5 business days for every SEV1/SEV2, with tracked corrective actions; recurring root causes feed back into this standard.

Area	Standard
AI safety incidents	Confirmed harmful or ungrounded clinical output is SEV1/SEV2 by impact: the offending model/prompt/ontology version is rolled back via kill switch (§14), the eval suite is extended with the failing case before re-release.
Change correlation	Deploys, flag flips, model/ontology promotions, and infra changes appear on one timeline so incidents correlate to changes in minutes.

19 Third-Party Dependencies & Licensing [NEW 3.0]

Area	Standard
License allowlist	Permitted without review: MIT, BSD, Apache-2.0, ISC. Weak copyleft (MPL, LGPL) requires architecture review. Strong copyleft (GPL, AGPL) and unknown/no-license packages are prohibited in shipped code. CI license scanning enforces this.
New dependencies	Adding a runtime dependency requires reviewer sign-off on maintenance health, security history, and transitive footprint — recorded in the PR.
Forks & vendoring	Forked or vendored packages documented with upstream reference and an owner tracking upstream security patches.
Content licensing	Ingested knowledge content (guidelines, ebooks, research) must have a documented licence permitting storage, chunking, embedding, and quotation; provenance recorded per document (§12.5).
SaaS & subprocessors	Any external service touching T1/T2 data requires security review before integration and a signed BAA where PHI is involved; subprocessor list reviewed periodically per SOC 2.
SDK hygiene	Analytics and crash SDKs configured with data collection minimized and verified against §9 (no T1/T2 leakage).

20 Enforcement

Clause	Rule
Vendor acceptance	Non-compliance with any requirement in this document is grounds for immediate PR rejection, milestone payment holdback, or mandatory remediation at the vendor's cost.
Non-functional requirements	Performance targets (§4, §6.2), coverage and eval gates (§12.6, §17.2), and security controls (§7–§10) are enforced identically to functional requirements.
Grounding requirements	The §12.6 anti-hallucination contract is a release gate: an assistant build that cannot demonstrate citation coverage, refusal behaviour, and contraindication safety is not shippable.
Traceability	Rules trace to the 27-item Change Register, the Rev 3.0 gap review, the Rev 3.1 conversational-data review, or the Rev 3.2 micro-frontend review; disputes resolve against the stated rationale.
Exceptions	Deviations require a written, time-boxed exception approved by the platform owner and recorded as an ADR — silent exceptions do not exist.